

ADR-006: Adoption of Drizzle as the ORM

Context

Interaction with our PostgreSQL database requires a robust, performant tool that is perfectly integrated into our TypeScript ecosystem. We need an ORM that guarantees type safety while avoiding the performance overhead of heavy execution engines, and that aligns well with our modern backend architectures.

Considered Options

- **Use Prisma:** A comprehensive tool we are familiar with, often the default choice in stacks with NestJS. It offers an excellent developer experience, but relies on a Rust engine (external binary) which can introduce performance overhead and slow down cold starts.
- **Use TypeORM:** A classic and mature solution, but heavy, tightly coupled with decorators, and sometimes generating complex and suboptimal SQL queries.
- **Adopt Drizzle ORM:** A lightweight, next-generation ORM with no external engine, offering a syntax very close to SQL while ensuring strict end-to-end typing in TypeScript.

Decision

Selected Decision: Adopt Drizzle as the ORM.

Justification: Drizzle integrates perfectly with our choice to strictly use TypeScript. It allows us to interact with our PostgreSQL database transparently ("if you know SQL, you know Drizzle") without the opaque abstraction layer of Prisma. Its lack of an external execution engine (zero-dependency runtime) guarantees excellent performance. It is the most pragmatic choice to retain full control over our schemas and query optimization.

Consequences

- **Positive:** Excellent synergy with TypeScript (strict typing), reducing runtime errors.
- **Positive:** High performance and lightweight (no external binary to bundle).
- **Positive:** Total control and predictability of the generated SQL queries.
- **Negative:** Requires better SQL mastery than a traditional ORM, and the ecosystem is still growing compared to older solutions.